# Modules & Plugin API coverage

YaPcore is **not** bundling MineMod. It exposes APIs so *you* can drop in plugins and fine-tune **modules** the same way.

## Three loadable kinds

| Kind | Folder | Manifest | Base class |
|---|---|---|---|
| Legacy Paper/Spigot | `plugins/` | `plugin.yml` | `org.bukkit.plugin.java.JavaPlugin` |
| YaP plugin | `plugins/` | `yap.yml` | `com.yapcore.api.YaPPlugin` |
| **Module** (fine-tune) | `modules/` | `module.yml` | `com.yapcore.api.module.YaPModule` |

Paper and YaP plugin jars share **`plugins/`** (see PLUGIN_COMPAT.md). Modules stay in `modules/`.

**Owner fine-tune path:** drop first-party packaging modules into `modules/` so every product surface is discoverable (`provides` / `requires`, Modules GUI, `FINE_TUNE.txt`). Engines and YAML stay in `plugins/` and `config/` — modules do not reimplement tick/economy logic.

```
gradle installProductDefaults      # CORE plugins + CORE fine-tune modules
gradle installFineTuneModules      # all packaging modules → modules/
gradle assemblePluginDist          # …/modules/core + …/modules/gameplay
```

See `modules/README.md` for the full jar table.

Central configs: TUNE.md. Purpur-class mob encyclopedia is the Paper plugin `yap-gameplay-knobs` (`plugins/YaPGameplayKnobs/knobs.yml`) plus packaging module `provides: [gameplay-knobs]` (GAMEPLAY tier).

**Vehicles:** GAMEPLAY opt-in — Paper plugin `yap-vehicles` + module `provides: [vehicles]`. See VEHICLES.md. Control from GUI, **web dashboard**, or `/yapvehicle`.

**Stacker:** Paper plugin `yap-stacker` + optional module `provides: [stacker]`. See STACKER.md (`/yapstacker`).

GUI tabs: **Plugins**, **Modules**, and **Tune**. Headless: WEB_DASHBOARD.md.

### Example `module.yml`

```
name: SpawnTweaks
main: com.example.SpawnTweaksModule
```

```
version: 1.0.0.0
api: yap-module-1
author: You
description: Optional spawn radius / MOTD tweaks
provides: [spawn-tweaks]
requires: []
```

Modules can declare `provides` / `requires` so operators compose only what they need.

## Threading contract (multithreaded server)

| Work | Pool | How |
|---|---|---|
| Inventory, blocks, teleport, world | **SYNC** | `Bukkit.getScheduler().runTask` or `getScheduler().runSync` |
| DB / HTTP / files / proxy sync | **HEAVY** | `runTaskAsynchronously` or `runHeavy` |
| Menu polish / animations | **UI** | `getScheduler().runUi` (YaP plugins/modules) |

`ThreadPools` tags the current thread. Bridge drain runs as SYNC. World/inv mutations auto-queue to the Compatibility Bridge when called off-SYNC.

**Race rule for plugin authors:** never mutate world/inventory from HEAVY/UI without hopping to SYNC. Keep caches labeled by owning pool.

## API coverage (what authors can use)

**Paper / Folia plugins (`plugin.yml`): Folia-native first-party** under product `game-authority=folia` (`folia-supported` + `YapSched`). Stock Paper jars are unsupported. Legacy `game-authority=paper` still exposes complete Paper API for benches — see PAPER_API_COVERAGE.md.

Runtime matrix: `com.yapcore.api.ApiCoverage`.

**YaP plugins / modules:** Adventure, dual-pool schedulers, modules `provides` / `requires`.

**Folia `RegionScheduler` APIs:** supported on the Folia product path via Folia + YapSched.

Runtime matrix: `com.yapcore.api.ApiCoverage` (see source).

> **Retired:** Paperclip / Phase 3 vendor scripts were removed. Use `./scripts/fetch-folia.sh` and `./scripts/build-yap-folia.sh` on the Folia product path.

## Author checklist

1. Put world/inv changes in `runTask` / `runSync`.
2. Put SQL/HTTP in async / `runHeavy`.
3. Prefer Adventure for text; legacy `§` strings still work.
4. Use modules for optional features operators can toggle by adding/removing jars.
5. Test on YaPcore; keep a Paper jar only if you also ship a Paper backend.

## Related

- PLUGINS.md — YaPPlugin dual-pool examples
- `examples/yap-allinone` — sample YaP plugin
- `examples/yap-module-demo` — sample module
- VEHICLES.md — vehicle API for Paper plugins
- `examples/yap-vehicle-addon` — sample third-party vehicle type
- `yap-first-party/modules/finetune-modules/` — first-party packaging modules source